Лабораторная работа № 8.

Контейнеризация REST API и базы данных

Тема: REST API товаров с SQL CRUD в Nginx, Express.js и PostgreSQL под Docker Compose

Цель: Научиться контейнеризировать REST API с PostgreSQL, выполнять CRUD через SQL, диагностировать многоконтейнерное приложение и сохранять данные в Docker Volume.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №8. Контейнеризация REST API и базы данных. Продолжительность: 4 академических часа (180 минут). Среда: Debian 13, Docker Engine, Docker Compose, Node.js 22 внутри контейнера, PostgreSQL 17, Nginx. Продолжаем архитектуру лабораторной №7, но создаём отдельный каталог проекта и отдельный том БД, чтобы не менять предыдущие данные.

2. Тема и цель

Тема: полноценный REST API товаров с SQL CRUD через Express.js и PostgreSQL, запускаемый средствами Docker Compose. Цель: собрать образ API, реализовать GET/POST/PATCH/DELETE, связать три контейнера и подтвердить постоянное хранение данных PostgreSQL.

3. Планируемые результаты

1. Подготавливать структуру проекта и файл compose.yaml.
2. Создавать SQL-таблицу с ограничениями и первичным ключом.
3. Выполнять параметризованные SQL-запросы с pg.Pool.
4. Собирать Docker-образ Express API.
5. Проверять маршруты CRUD через curl и данные через psql.
6. Диагностировать ошибки связи с БД и сохранять данные при пересоздании контейнеров.

4. Оборудование и исходные данные

Нужны: виртуальная машина Debian 13 с установленными Docker Engine и Docker Compose, терминал с sudo, доступ к реестру образов. Node.js на хосте устанавливать не требуется. Команды выполняются в Debian. Используются только учебные пароли: не переносите их на реальный сервер.

Параметры учебного приложения

Компонент	Значение
Каталог	compose-course8
Имя проекта Compose	course8 (через -p course8)
Внешний URL	http://127.0.0.1:8080/api/products
Сервисы	nginx, api, db
PostgreSQL	postgres:17, база shop, пользователь shopuser
Порт БД	5432 только внутри сети

Компонент	Значение
Docker Volume	course8_pgdata (фактическое имя при проекте course8)
Таблица	products(id, name, price)

5. Архитектура и структура файлов

Схема взаимодействия

```
Браузер / curl
  | HTTP :8080
  v
nginx (:80) -> api (:3000) -> db (:5432)
                                |
                                v
                      pgdata (Docker Volume)
```

Файлы проекта

```
compose-course8/
├── compose.yaml
├── db/
│   └── init.sql
├── nginx/
│   └── default.conf
└── api/
    ├── app.js
    ├── package.json
    ├── package-lock.json
    ├── Dockerfile
    ├── .dockerignore
    ├── config/db.js
    ├── routes/productRoutes.js
    ├── controllers/productController.js
    └── services/productService.js
```

6. Подготовка учебной среды

Проверка Docker

```
sudo docker version
sudo docker compose version
```

Проверьте наличие секций Client/Server и вывод версии Compose. Если Docker не установлен, выполните предварительную лабораторную №2. Не продолжайте до устранения ошибок запуска демона.

Создание каталогов

```
mkdir -p compose-course8/api/{config,routes,controllers,services} compose-
course8/{db,nginx}
cd compose-course8
```

Критерий: каталоги api, db, nginx существуют. Все следующие пути относительны к compose-course8.

7. Создание SQL-таблицы

db/init.sql

```
CREATE TABLE IF NOT EXISTS products (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price NUMERIC(12,2) NOT NULL CHECK (price >= 0)
);

INSERT INTO products (name, price) VALUES
  ('Keyboard', 15000),
  ('Mouse', 8000);
```

Файл будет выполнен официальным образом postgres:17 только при первоначальной инициализации пустого каталога PGDATA. При повторном старте существующего тома данные не перезаписываются. Создайте проект с новым томом, как указано ниже; не используйте down -v для сохранённых данных.

8. Подготовка приложения Express

api/package.json

```
{
  "name": "course8-api",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^5.1.0",
    "pg": "^8.16.0"
  }
}
```

Создайте package.json. Для формирования package-lock.json потребуется npm. Если на Debian npm не установлен, временно запустите официальный контейнер Node.js с монтированием рабочей папки:

Создание package-lock.json без установки Node.js на хост

```
sudo docker run --rm -v "$PWD/api:/app" -w /app node:22-alpine npm install --package-lock-only
```

После выполнения убедитесь, что появился api/package-lock.json. Образ Node скачивается только при первом обращении; команда работает без установки Node.js на хосте. Версии зависимостей фиксируются в lockfile, поэтому npm ci в Dockerfile будет воспроизводимым.

api/config/db.js

```
import pg from 'pg';

export const pool = new pg.Pool({
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT || 5432),
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});
```

В Docker Compose хост базы данных — имя сервиса db. Указание localhost означало бы сам контейнер API, а не контейнер PostgreSQL.

api/services/productService.js

```
import { pool } from '../config/db.js';

export async function getAll() {
  const { rows } = await pool.query('SELECT id, name, price FROM products ORDER BY id');
  return rows;
}

export async function getById(id) {
  const { rows } = await pool.query('SELECT id, name, price FROM products WHERE id=$1',
[id]);
  return rows[0] || null;
}

export async function create({ name, price }) {
  const { rows } = await pool.query(
    'INSERT INTO products (name, price) VALUES ($1, $2) RETURNING id, name, price',
    [name, price]
  );
  return rows[0];
}

export async function update(id, changes) {
  const current = await getById(id);
  if (!current) return null;
  const name = changes.name ?? current.name;
  const price = changes.price ?? current.price;
  const { rows } = await pool.query(
    'UPDATE products SET name=$1, price=$2 WHERE id=$3 RETURNING id, name, price',
    [name, price, id]
  );
  return rows[0] || null;
}

export async function remove(id) {
  const { rowCount } = await pool.query('DELETE FROM products WHERE id=$1', [id]);
  return rowCount > 0;
}
```

Сервис выполняет SQL-запросы, все значения передаются отдельно через параметры $1, $2. Использовать конкатенацию строк для вставки пользовательских данных в SQL запрещается.

api/controllers/productController.js

```
import * as service from '../services/productService.js';

function idFromRequest(req) {
  const id = Number(req.params.id);
  return Number.isSafeInteger(id) && id > 0 ? id : null;
}

function valid(body, partial = false) {
  if (!body || typeof body !== 'object' || Array.isArray(body)) return false;
  const keys = Object.keys(body);
  if (keys.some(k => !['name', 'price'].includes(k))) return false;
  if (partial && keys.length === 0) return false;
  if (!partial && (!keys.includes('name') || !keys.includes('price'))) return false;
  if ('name' in body && (typeof body.name !== 'string' || !body.name.trim() ||
body.name.length > 100)) return false;
  if ('price' in body && (typeof body.price !== 'number' || !Number.isFinite(body.price)
|| body.price < 0 || Math.round(body.price * 100) !== body.price * 100)) return false;
  return true;
}

export async function getAll(req, res) {
  res.json(await service.getAll());
}

export async function getById(req, res) {
  const id = idFromRequest(req);
  if (id === null) return res.status(400).json({ message: 'Некорректный ID' });
  const product = await service.getById(id);
  if (!product) return res.status(404).json({ message: 'Товар не найден' });
  res.json(product);
}

export async function create(req, res) {
  if (!valid(req.body)) return res.status(400).json({ message: 'Некорректный товар' });
  const product = await service.create({ name: req.body.name.trim(), price:
req.body.price });
  res.status(201).json(product);
}

export async function update(req, res) {
  const id = idFromRequest(req);
  if (id === null) return res.status(400).json({ message: 'Некорректный ID' });
  if (!valid(req.body, true)) return res.status(400).json({ message: 'Некорректные
данные' });
  const changes = { ...req.body };
  if ('name' in changes) changes.name = changes.name.trim();
  const product = await service.update(id, changes);
  if (!product) return res.status(404).json({ message: 'Товар не найден' });
  res.json(product);
}

export async function remove(req, res) {
  const id = idFromRequest(req);
  if (id === null) return res.status(400).json({ message: 'Некорректный ID' });
  const deleted = await service.remove(id);
  if (!deleted) return res.status(404).json({ message: 'Товар не найден' });
  res.status(204).send();
}
```

Controller проверяет тело и ID; Service не знает об HTTP. В учебном примере цена передаётся числом, а PostgreSQL NUMERIC обычно читается через pg как строка в JSON. Это особенности контракта данных, не ошибка.

api/routes/productRoutes.js

```
import { Router } from 'express';
import * as controller from '../controllers/productController.js';

const router = Router();
router.get('/', controller.getAll);
router.get('/:id', controller.getById);
router.post('/', controller.create);
router.patch('/:id', controller.update);
router.delete('/:id', controller.remove);
export default router;
```

api/app.js

```
import express from 'express';
import productRoutes from './routes/productRoutes.js';

const app = express();
app.use(express.json());
app.use('/api/products', productRoutes);
app.use((err, req, res, next) => {
  console.error(err);
  if (err.type === 'entity.parse.failed') return res.status(400).json({ message:
'Некорректный JSON' });
  res.status(500).json({ message: 'Ошибка сервера' });
});

app.listen(3000, '0.0.0.0', () => console.log('API: port 3000'));
```

Express 5 передаёт ошибки отклонённых Promise в error middleware. Внешний доступ к API будет организован через Nginx, не через публикацию порта контейнера API.

9. Сборка Docker-образа

api/Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

api/.dockerignore

```
node_modules
npm-debug.log
.git
.env
```

npm ci использует lockfile. Файл .dockerignore исключает локальные зависимости и секреты из контекста сборки. Старайтесь не выполнять команды Docker под root внутри контейнера в реальных средах; здесь приоритет — освоение CRUD и Compose.

10. Настройка Nginx и Compose

nginx/default.conf

```
server {
    listen 80;
    location / {
        proxy_pass http://api:3000;
    }
}
```

compose.yaml

```
services:
  nginx:
    image: nginx:stable
    ports:
      - '8080:80'
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api
  api:
    build: ./api
    environment:
      DB_HOST: db
      DB_PORT: '5432'
      DB_USER: shopuser
      DB_PASSWORD: training_only
      DB_NAME: shop
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:17
    environment:
      POSTGRES_USER: shopuser
      POSTGRES_PASSWORD: training_only
      POSTGRES_DB: shop
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U shopuser -d shop']
      interval: 5s
      timeout: 3s
      retries: 10
volumes:
  pgdata:
```

Здесь публикуется только порт Nginx. Сервисы api и db доступны по DNS-именам внутри стандартной сети проекта Compose. Если порт 8080 занят, сначала выявите и остановите конфликтующий учебный сервис, либо по согласованию с преподавателем измените публикацию порта.

Валидация YAML и запуск

```
sudo docker compose -p course8 config
sudo docker compose -p course8 up -d --build
sudo docker compose -p course8 ps
```

```

```

Критерий: nginx и api в состоянии Up, db в состоянии healthy. Первый старт может занять время из-за загрузки образов и инициализации БД. Ошибки смотрите через docker compose logs.

11. Проверка SQL и REST CRUD

Проверка исходных записей

```
sudo docker compose -p course8 exec db psql -U shopuser -d shop -c "SELECT * FROM products ORDER BY id;"
```

Ожидаются две строки Keyboard и Mouse. Если таблицы нет, проверьте, не был ли ранее создан непустой том; init.sql не применяется повторно к существующему каталогу данных.

GET: список товаров

```
curl -i http://127.0.0.1:8080/api/products
```

Ожидается HTTP 200 и JSON-массив. Поле price может быть строкой вида "15000.00": это стандартное преобразование PostgreSQL NUMERIC в драйвере pg.

POST: создание товара

```
curl -i -X POST http://127.0.0.1:8080/api/products \
  -H 'Content-Type: application/json' \
  -d '{"name":"SSD","price":42000}'
```

Ожидается 201 Created и объект с новым id. Запишите присвоенный id и используйте его на следующих шагах. Если в базе были иные записи, фактический id может быть не равен 3.

GET: товар по id

```
curl -i http://127.0.0.1:8080/api/products/1
```

Ожидается HTTP 200 и объект товара Keyboard при исходной базе. Для несуществующего id должен возвращаться 404.

PATCH: изменение цены существующего товара

```
curl -i -X PATCH http://127.0.0.1:8080/api/products/1 \
  -H "Content-Type: application/json" \
  -d "{\"price\":13000}"
```

Ожидается HTTP 200 и обновлённое значение цены. В этой команде кавычки JSON экранированы для оболочки bash. При изменении только цены название должно сохраниться.

DELETE: удаление товара

```
curl -i -X DELETE http://127.0.0.1:8080/api/products/2
```

```

```

Ожидается 204 No Content. Повторный GET /api/products/2 должен вернуть 404. Удалённая запись остаётся удалённой после перезапуска API благодаря PostgreSQL.

12. Диагностика ошибок

Некорректный товар

```
curl -i -X POST http://127.0.0.1:8080/api/products \
  -H 'Content-Type: application/json' \
  -d '{"name":"Bad","price":-1}'
```

Ожидается 400, запись в таблицу не добавляется. Объясните, зачем валидация применяется одновременно в API и в ограничении CHECK базы данных.

Команды диагностики

```
sudo docker compose -p course8 logs --tail=40 nginx
sudo docker compose -p course8 logs --tail=40 api
sudo docker compose -p course8 logs --tail=40 db
sudo docker compose -p course8 ps
```

По журналам определите, какой сервис возвращает ошибку. Важный признак: Nginx может возвращать 502, когда сервис api недоступен, а API может возвращать 500 при ошибке SQL.

Эксперимент: остановка базы

```
sudo docker compose -p course8 stop db
curl -i http://127.0.0.1:8080/api/products
sudo docker compose -p course8 start db
sudo docker compose -p course8 ps
```

После остановки БД API не сможет выполнить SELECT. Зафиксируйте код ошибки, затем восстановите db, дождитесь healthy и повторите GET. Не удаляйте том и не используйте down -v.

13. Проверка постоянного хранения

Пересоздание контейнеров без удаления тома

```
sudo docker compose -p course8 down
sudo docker compose -p course8 up -d
curl -i http://127.0.0.1:8080/api/products
```

Критерий: данные, созданные через POST и изменённые через PATCH, сохранились; удалённый товар не появился снова. down удаляет контейнеры, но не именованный том pgdata. Не применяйте down -v — оно удаляет том проекта.

14. Самостоятельное задание

7. Создайте через POST товар Monitor стоимостью 95000.
8. Получите товары через GET и запишите присвоенный id.
9. Измените через PATCH цену нового товара на 90000.

10. Удалите через DELETE один ранее созданный учебный товар, не затрагивая другие данные.
11. Перезапустите стек через down/up и докажите сохранность оставшихся записей SQL-запросом.

15. Ожидаемый результат

Работает проект из nginx, api, db; запросы к http://127.0.0.1:8080/api/products выполняют CRUD в PostgreSQL; все SQL-данные сохраняются в именованном томе проекта; команды docker compose ps, logs и exec помогают диагностировать неисправности.

16. Содержание отчёта

12. Тема, цель, сведения о среде (версии Docker и Compose).
13. Структура каталогов и содержимое compose.yaml.
14. SQL-схема таблицы products и результат SELECT.
15. Результаты и HTTP-статусы GET/POST/PATCH/DELETE, включая ответы об ошибках.
16. Логи и описание диагностического эксперимента с остановкой db.
17. Подтверждение сохранности данных после down/up.
18. Вывод о разделении обязанностей между nginx, api и db.

17. Контрольные вопросы

19. Почему клиент обращается к nginx, а не напрямую к db?
20. Почему DB_HOST=db, а не localhost?
21. Где фактически хранятся товары?
22. Что делает RETURNING в SQL?
23. Зачем нужны параметры $1 и $2 в SQL-запросах?
24. Как отличаются функции Router, Controller и Service?
25. Почему для POST используется статус 201?
26. Что означает статус 204 при DELETE?
27. Когда исполняется SQL-файл в docker-entrypoint-initdb.d?
28. Для чего нужен healthcheck PostgreSQL?
29. Какой командой проверить записи непосредственно в БД?
30. Чем отличаются docker compose down и down -v?

18. Критерии оценивания

Оценивание

Критерий	Баллы
Подготовка проекта, сборка и запуск Compose	15
SQL-таблица и проверка подключения	15
GET и POST товаров	20
PATCH и DELETE товаров	20
Диагностика и обработка ошибок	10
Сохранность данных и самостоятельное задание	10
Отчёт и ответы на вопросы	10
Итого	100